

Laplace 2D

CDC · H,G · solve

Lucas S. Campos

Introdução ao MEC / BEM

“Laplace 2D”

Geometria, N_k , J e $\mathbf{n}$ já vieram de **Indo para 2D**. Aqui o Gmsh deixa de ser só malha e passa a carregar **CDC**; montamos H , G e resolvemos o primeiro BEM 2D completo.

“Objetivos”

1. Rodar o fluxo mínimo do quadrado ($T = x$) no BEM_gmsh.
2. Ler CDCs nos **grupos físicos** ("0;T", "1;q") e ligar a `dad.BC` / `dad.BV`.
3. Seguir o pipeline PDE $\rightarrow$ equação integral $\rightarrow HT = Gq \rightarrow Ax = b \rightarrow$ internos.
4. Entender a diagonal de H (corpo rígido) e o papel de `npg` na montagem.

Código: BEM_gmsh. Na pasta do projeto:

```
using Pkg
Pkg.activate(".")
Pkg.instantiate()

using DrWatson
@quickactivate :BEM
include(datadir("Laplace", "Laplace_dad.jl"))
```

Se o capítulo **Indo para 2D** ainda não rodou, comece por `data/examples/geo_unit_square.jl`.

“Fluxo mínimo (quadrado,)”

Rode **antes** da álgebra — o resto do capítulo só nomeia cada linha.

```
props = Laplace(1.0)
msh    = quadrado(ndiv=20, show=false)
dad    = format2d(msh, props)

attach_analytical!(dad, ana_laplace_linear(; direction=SA[1.0, 0.0]))
H_G_full_direct(dad, 20) # npg = 20 (Gauss na montagem)
solve(dad)

@show rel_error(dad)
plot_geo(dad)
```

As CDCs já vêm dos grupos físicos do quadrado (ver abaixo). `attach_analytical!` **não** sobrescreve CDC: só guarda a solução exata para `rel_error`.

A **mesma** PDE descreve vários problemas: mudam o nome do campo, a lei constitutiva e o significado de f e do fluxo. Com $f = 0$ é Laplace; com fonte de domínio, Poisson (capítulo seguinte).

Equação	Problema físico	Grandezas	Lei / fluxo
$\nabla^2\varphi = 0$	Torção de Saint-Venant	φ : função de empenamento	Hooke (tensões a partir de φ)
$\nabla \cdot (S\nabla u) + f = 0$	Deflexão de membranas	u : deflexão; f : carga; S : tensão	$D = SI$
$\nabla \cdot (D\nabla T) + f = 0$	Condução de calor	T : temperatura; D : condutividade; f : geração	Fourier: $\mathbf{q} = -D\nabla T$
$\nabla \cdot (\nabla\varphi) + f = 0$	Escoamento potencial (irrot., invíscido)	φ : potencial de velocidade; f : fonte	$\mathbf{v} = \nabla\varphi$
$\nabla \cdot (D\nabla\varphi) + f = 0$	Meio poroso	φ : carga piezométrica; D : permeabilidade	Darcy: $\mathbf{q} = -D\nabla\varphi$
$\nabla \cdot (D\nabla V) + f = 0$	Potencial elétrico	V : potencial; D : condutividade elétrica	Ohm: $\mathbf{q} = -D\nabla V$

Tabela 1: Exemplos de problemas físicos descritos por Laplace ou Poisson.

No curso. O protótipo do BEM_gmsh é potencial T com $q = -k\partial T/\partial n$ (glossário). Anisotropia ou $f \neq 0$ mudam a SF ou pedem termo de domínio (Poisson / DIBEM). As matrizes H, G seguem a **mesma** lógica: campo no contorno + fluxo conjugado.

No Gmsh, o **nome** da curva física carrega tipo e valor. O `format2d` lê isso e preenche `dad.BC` (0 = Dirichlet, 1 = Neumann) e `dad.BV` (valor).

Nome	Significado (Laplace)
"0;T"	Dirichlet: T prescrito
"1;q"	Neumann: $q = -k\partial T/\partial n$ prescrito

Exemplos: "0;100", "1;0" (isolado). Elasticidade: "tx;ux;ty;uy" (capítulo próprio).

“O quadrado do pacote (CDCs de)”

Em `data/Laplace/Laplace_dad.jl`, com $k = 1$ e solução $T = x$ ($q = -\partial T / \partial n$):

```
# Laplace_dad.jl – trecho de quadrado(...)
# l1 bottom, l2 right, l3 top, l4 left
gmsh.model.addPhysicalGroup(1, [l1, l3], -1, "1;0")    # isolado (q = 0)
gmsh.model.addPhysicalGroup(1, [l2], -1, "1;-1")      # direita: q = -1
gmsh.model.addPhysicalGroup(1, [l4], -1, "0;0")        # esquerda: T = 0
gmsh.model.addPhysicalGroup(2, [s1], -1, "Domain")
```

Conferência rápida: normal saindo à direita é $+\mathbf{e}_x$, $\partial T / \partial n = 1$, logo $q = -1$; topo/base $\partial T / \partial n = 0$; esquerda $T = 0$.

“Cantos e elemento descontínuo”

Em um vértice, duas arestas podem pedir CDCs **incompatíveis** no mesmo nó (ex.: T de um lado e q do outro, ou dois T diferentes). Por isso o `format2d` coloca nós de **campo** no interior do elemento (`discontinuous_nodes_weights` / Gauss–Legendre) — cada elemento tem seus T_i, q_i , sem nó compartilhado no canto. A geometria pode continuar isoparamétrica nos vértices (Equispaced), como no capítulo **Indo para 2D**.

```
# Analytical.jl (essência)
function attach_analytical!(dad::BEMdata, ana::AnalyticalSolution)
    set_cache!(dad; analytical=ana)
    return dad
end
```

Só registra a referência para erro / gráficos. Para **forçar** Dirichlet analítico em todos os nós (patch test), o pacote tem `apply_analytical_bc!` (`dad, ana`) — isso **sim** sobrescreve BC / BV. No fluxo didático padrão do quadrado, **não** chame `apply_analytical_bc!`: as CDCs do `.msh` já batem com $T = x$.

Cada seção seguinte preenche **um** passo. A diagonal de H é detalhe do passo 4.

1. PDE

$\nabla^2 T = 0$ em Ω , CDCs em Γ

⇓

2. Resíduos + Green

o operador passa de T para o peso v

⇓

3. Solução fundamental

$v = T^*$ com $-\nabla^2 T^* = \delta(x - x_d)$; o domínio some $\rightarrow$ equação integral só no contorno

⇓

4. Discretização

$T, q \approx N_k$; integrais $\rightarrow H, G$ (quadratura npg, singularidades, **diagonal de H**)

⇓

5. CDC

em cada nó, T **ou** q conhecido $\rightarrow HT = Gq$ vira $Ax = b$

⇓

6. Solve

$x = A^{-1}b \rightarrow (T, q)$ completo no contorno

⇓

7. Internos

com contorno conhecido, $T(x_d)$ interior é **só** integral — sem novo sistema linear

Passo	No BEM_gmsh
1–3 formulação	fundamental / núcleos na montagem
4 H, G	H_G_full_direct(dad, npg)
5 CDC	grupos Gmsh + format2d; troca de colunas em applyBC
6 solve	solve(dad) $\rightarrow$ applyBC + bem_linsolve
7 internos	pontointerno=true em format2d; plot_geo

Ideia-chave. No MEF montamos “rigidez” no **volume**. No BEM montamos H e G no **contorno**; o preço é matrizes cheias e integrais singulares quando a fonte x_d cai no elemento integrado.

“Passos 1-3: da PDE à equação integral”

Problema canônico do capítulo:

$$\nabla^2 T = 0 \quad \text{em } \Omega,$$

com $q = -k\partial T/\partial n$ no contorno (quando $f \neq 0$, ver **Poisson 2D**).

Em cinco movimentos:

1. Resíduo ponderado: $\int_{\Omega} v \nabla^2 T \, d\Omega = 0$ para pesos v adequados.
2. Integração por partes / 2ª identidade de Green ($u = T$, peso v):

$$\int_{\Omega} (v \nabla^2 T - T \nabla^2 v) \, d\Omega = \int_{\Gamma} (v \partial_n T - T \partial_n v) \, ds.$$

3. Escolhe-se a **solução fundamental** $v = T^*$ tal que $-\nabla^2 T^* = \delta(x - x_d)$ (em sentido distribucional).
4. O termo de domínio com $\nabla^2 T^*$ vira avaliação em x_d ; com $\nabla^2 T = 0$, sobra uma **equação integral só em** Γ .
5. No contorno liso o coeficiente de $T(x_d)$ é $1/2$ (ângulo sólido); no interior é 1 ; em cantos, $c(x_d) \neq 1/2$ – por isso o código prefere elementos descontínuos ou calcula a diagonal de H de forma **indireta** (passo 4).

Equação integral no contorno liso:

$$c(x_d)T(x_d) = \int_{\Gamma} T q^* \, ds - \int_{\Gamma} T^* q \, ds, \quad c = 1/2 \quad (\text{liso}) \, .$$

No pacote, os núcleos 2D ($U \triangleq T^*$, $T \triangleq q^*$ no código) são:

```
# Laplace/Fundamental.jl
function fundamental(props::Laplace, r::SVector{2}, n::SVector{2})
    k = props.k
    R = _R(r)                                # |r|
    G = -log(R) / (2π * k)                   # T*
    H = dot(r, n) / (R^2 * 2π)               # q*
    return KernelPair(G, H)
end
```

Ou seja (notação das notas):

$$T^* = -(\ln r)/(2\pi k), \quad q^* = (\mathbf{r} \cdot \mathbf{n})/(2\pi r^2).$$

“Passo 4: discretização e”

Em cada elemento descontínuo com m nós:

$$T(\xi) = \sum_{k=1}^m N_k(\xi) T_k, \quad q(\xi) = \sum_{k=1}^m N_k(\xi) q_k.$$

Colocando a fonte em cada nó i e somando elementos $j = 1, \dots, n_{\text{elem}}$:

$$\sum_j \sum_k \left(\int_{\Gamma_j} q^* N_k \, d\Gamma \right) T_k^j - \sum_j \sum_k \left(\int_{\Gamma_j} T^* N_k \, d\Gamma \right) q_k^j = 0 \quad (+ \text{ termo livre na diagonal de } H) .$$

Em forma matricial global:

$$HT = Gq.$$

Definições operacionais (linha i = fonte no nó i ; coluna ligada ao GL k):

$$H_{ik} = \int_{\Gamma} q^*(x, x_i) N_k(x) \, d\Gamma + c_i \delta_{ik}, \quad G_{ik} = \int_{\Gamma} T^*(x, x_i) N_k(x) \, d\Gamma.$$

- H : núcleo **mais** singular ($q^* \sim 1/r$ em 2D no elemento da fonte).
- G : singularidade **fraca** ($\ln r$), em geral integrável com quadratura adequada.
- `npg` em `H_G_full_direct(dad, npg)` fixa a quadratura de montagem (eco do Gauss do cap. 05).

Trecho real da montagem densa (`src/Core/Assembly_full.jl`):

```
function H_G_full_direct(dad::BEMdata{<:Scalar}; npg=20, threaded=true)
    _init_quadrature!(dad, npg)
    H = zeros(dad.nt, dad.nt)
    G = zeros(dad.nt, dad.n)
    set_cache!(dad; H, G)
    # ... para cada fonte i e elemento el:
    #     integrate_element(...) ou _far_nodal_scalar!(...)
    #     H[i, j] += ...; G[i, j] += ...
    @inbounds for i in 1:dad.nt
```

```
H[i, i] = 0.0
H[i, i] = -sum(H[i, :])    # diagonal por soma nula (corpo rígido)
end
return H, G
end
```

No elemento singular o pacote usa Guiggiani; no quase-singular, regra sinh; longe, lumping nodal — detalhes em `integrate_element`. Para a aula, o essencial é: **integra o que for regular e fecha a diagonal de H por identidade.**

“Diagonal de : corpo a temperatura constante”

Quando a fonte está no mesmo elemento, integrar H_{ii} “na marra” é delicado. Em vez disso, usa-se a solução exata trivial de Laplace

$$T(x) \equiv 1, \quad q(x) \equiv 0.$$

Ela implica $H\{1\} = G\{0\} = \{0\}$, logo **cada linha de H soma zero**. Com os H_{ij} ($i \neq j$) já calculados,

$$H_{ii} = - \sum_{j \neq i} H_{ij}.$$

Isso **já inclui** o fator de ângulo sólido ($1/2$ no liso, outro valor em cantos). No código: exatamente o laço `H[i,i] = -sum(H[i,:])` acima.

Ordem prática (passo 4):

1. zere H e G ;
2. para cada par (fonte i , elemento j), some contribuições regulares / tratadas;
3. $H_{ii} = - \sum_{j \neq i} H_{ij}$;
4. diagonal de G : em geral **inteiro** (singularidade fraca) — o truque indireto raramente é necessário.

“Passos 5-6: CDC e”

Depois de $HT = Gq$ ainda falta, em cada nó, **uma** informação. A CDC decide qual coluna vai para a esquerda (incógnita) e o que alimenta b .

Regra por nó i :

- **Dirichlet** (T_i conhecido, $BC[i]==0$): incógnita $q_i \rightarrow$ coluna i de $-G$ em A ; $H_{:,i}T_i$ vai a b ;
- **Neumann** (q_i conhecido, $BC[i]==1$): incógnita $T_i \rightarrow$ coluna i de H em A ; $G_{:,i}q_i$ vai a b .

No pacote (src/Core/Boundary_conditions.jl):

```
function applyBC(dad::BEMdata{<:Union{Laplace,OrthotropicLaplace}}, A, B, b)
    n = dad.n
    for bc in 1:n
        if dad.BC[bc] == 0          # Dirichlet: incógnita = q
            A[:, bc] .= -.view(dad.G, :, bc)
            B[:, bc] .= -.view(dad.H, :, bc)
        end
    end
    fill!(b, 0)
    for j in 1:n
        if dad.BC[j] == 0
            b .-= view(dad.H, :, j) .* dad.BV[j]
        else
            b .+= view(dad.G, :, j) .* dad.BV[j]
        end
    end
end
```

e o solve estacionário:

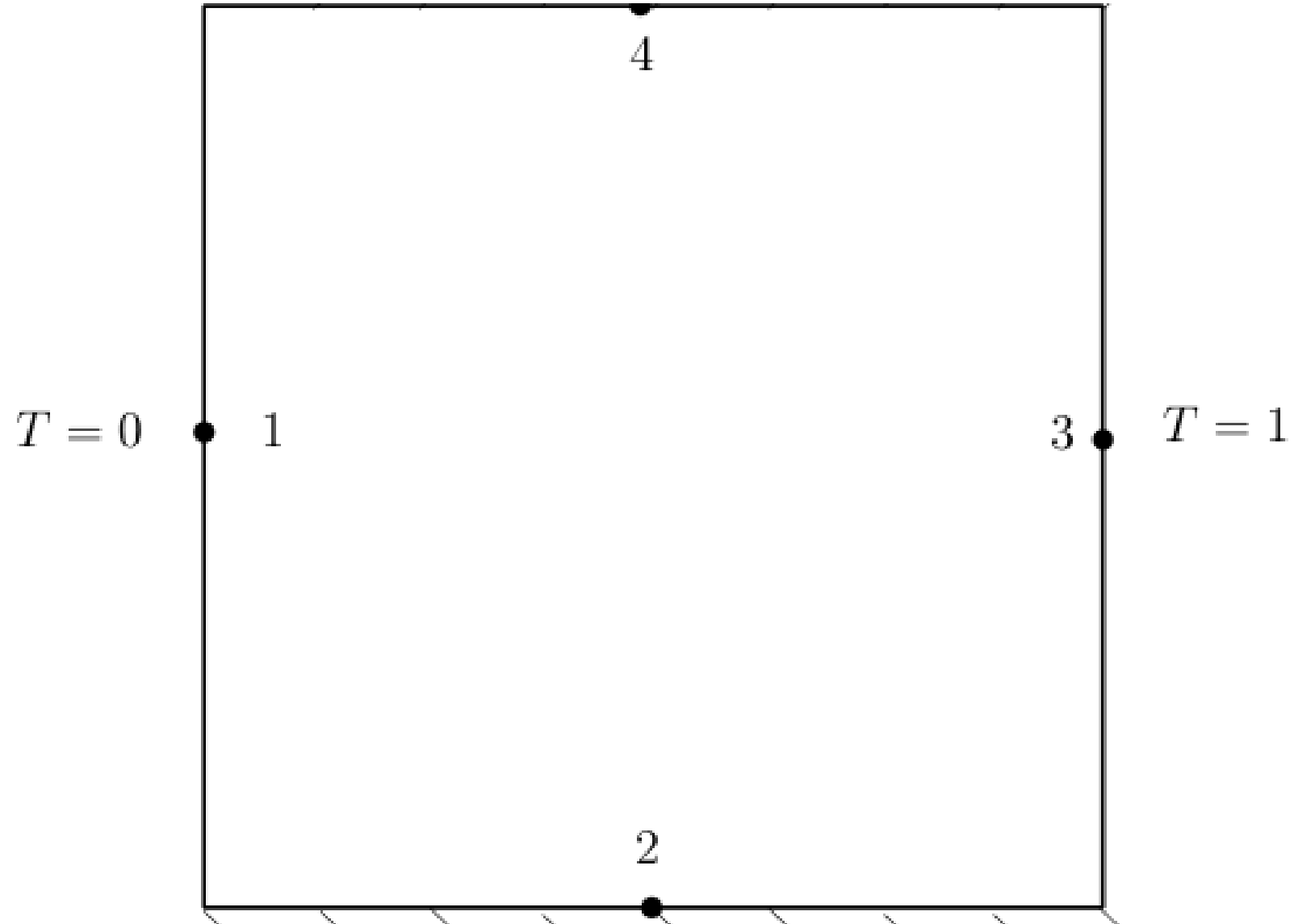
```
# Core/Solver.jl (essência)
function solve(dad::BEMdata{<:Union{Laplace,OrthotropicLaplace}}; ...)
    applyBC(dad; ...)
    x = bem_linsolve(dad.A, dad.b)
    # split_sol! → dad.T, dad.q
```

```
    return dad.T  
end
```

“Exemplo didático (1 elemento por lado)”

Condução unidimensional com um elemento por aresta do retângulo:

“Laplace 2D”



“Exemplo didático (1 elemento por lado)”

Esquema (macron = conhecido):

$$H \begin{pmatrix} \bar{T}_1 \\ T_2 \\ \bar{T}_3 \\ T_4 \end{pmatrix} = G \begin{pmatrix} q_1 \\ \bar{q}_2 \\ q_3 \\ \bar{q}_4 \end{pmatrix}.$$

Reorganizando colunas conhecidas / desconhecidas obtém-se $Ax = b$ com $x = (q_1, T_2, q_3, T_4)$. No quadrado real as CDCs misturam Dirichlet e Neumann como na seção dos grupos físicos — o mecanismo é o mesmo applyBC.

“Passo 7: pontos internos”

Para $x_d \in \Omega$ (fora de Γ), $c = 1$:

$$T(x_d) = \int_{\Gamma} T q^* \, ds - \int_{\Gamma} T^* q \, ds.$$

Com (T, q) **já conhecidos em todo o contorno**, isso é pós-processamento: **não** se monta nem se resolve outro $Ax = b$. Ative com `format2d(..., pontointerno=true)`.

“Lab guiado: quadrado”

Um único script (relatório em tela). CDCs = grupos do quadrado; referência = ana_laplace_linear.

```
using DrWatson
@quickactivate :BEM
using LinearAlgebra, Printf
include(datadir("Laplace", "Laplace_dad.jl"))

props = Laplace(1.0)
msh    = quadrado(ndiv=16, show=false)
dad    = format2d(msh, props; pontointerno=true)

println("N = ", dad.n, "  internos = ", length(dad.internalNodes))

attach_analytical!(dad, ana_laplace_linear(; direction=SA[1.0, 0.0]))
H_G_full_direct(dad, 16)
solve(dad)

@printf "rel_error = %.6e\n" rel_error(dad)

println(" i |    x |    y |  T_num | T_exato |    q_num")
for i in 1:min(8, dad.n)
    x, y = dad.Nodes[i]
    @printf "%2d | %5.3f | %5.3f | %7.5f | %7.5f | %8.5f\n" i x y dad.T[i] x dad.q[i]
end

eT = maximum(abs(dad.T[i] - dad.Nodes[i][1]) for i in 1:dad.n)
@printf "max |T_num - x| = %.6e\n" eT
plot_geo(dad)
```

Observar:

- rel_error cai com ndiv in {8,16,32} (tabela no exercício 0);
- lados verticais: $T \approx x$; horizontais: $q \approx 0$;

- internos seguem $T \approx x$ sem novo sistema.

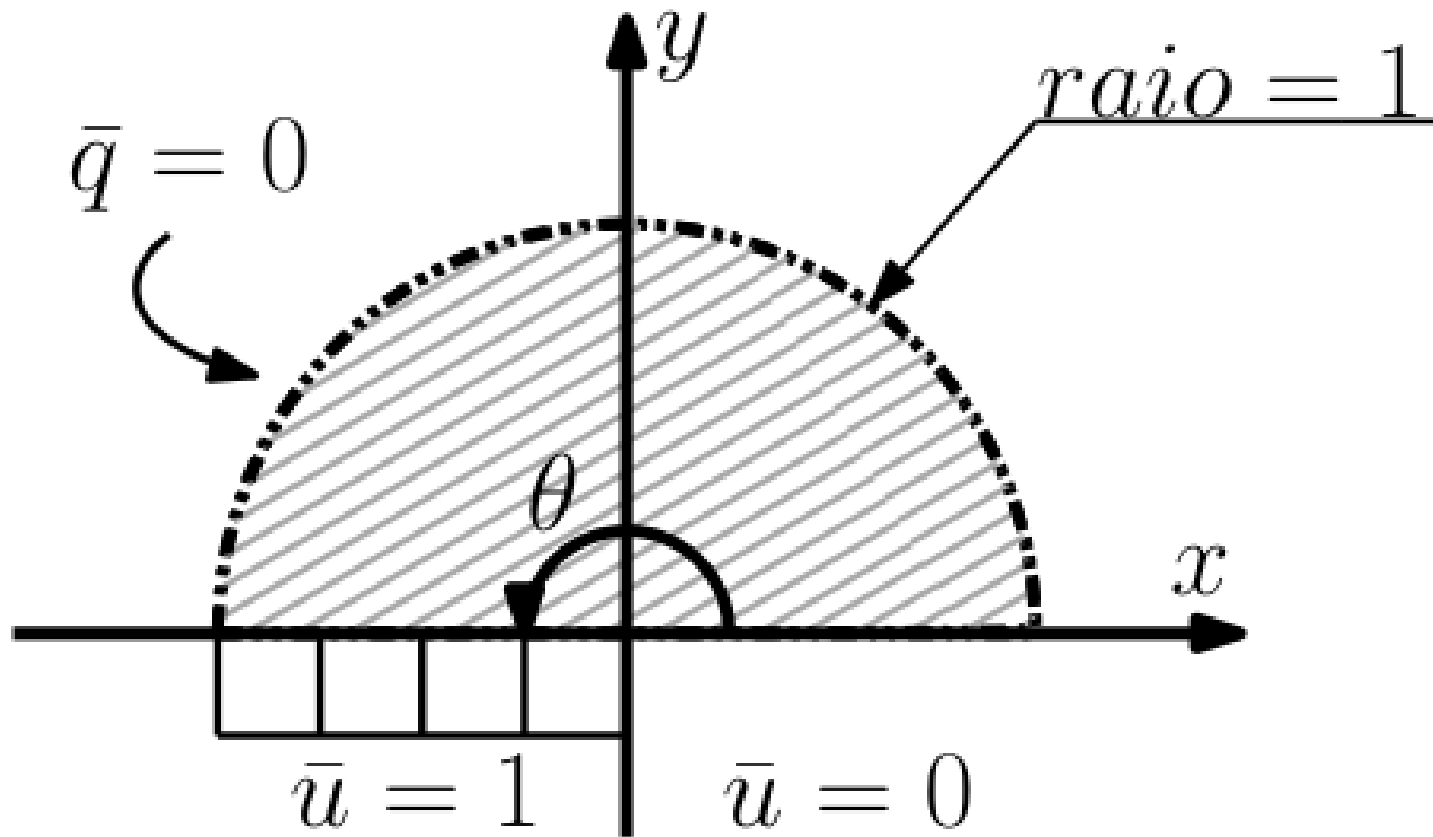
“Exercícios”

Notação: $T, q = -k\partial T/\partial n$. Erros: **Apêndice: medidas de erro** e/ou `rel_error(dad)`. Ordem do elemento: `tipo / ordem` no gerador e em `format2d` (linear $p = 1$, quadrático $p = 2$, ...).

Escada

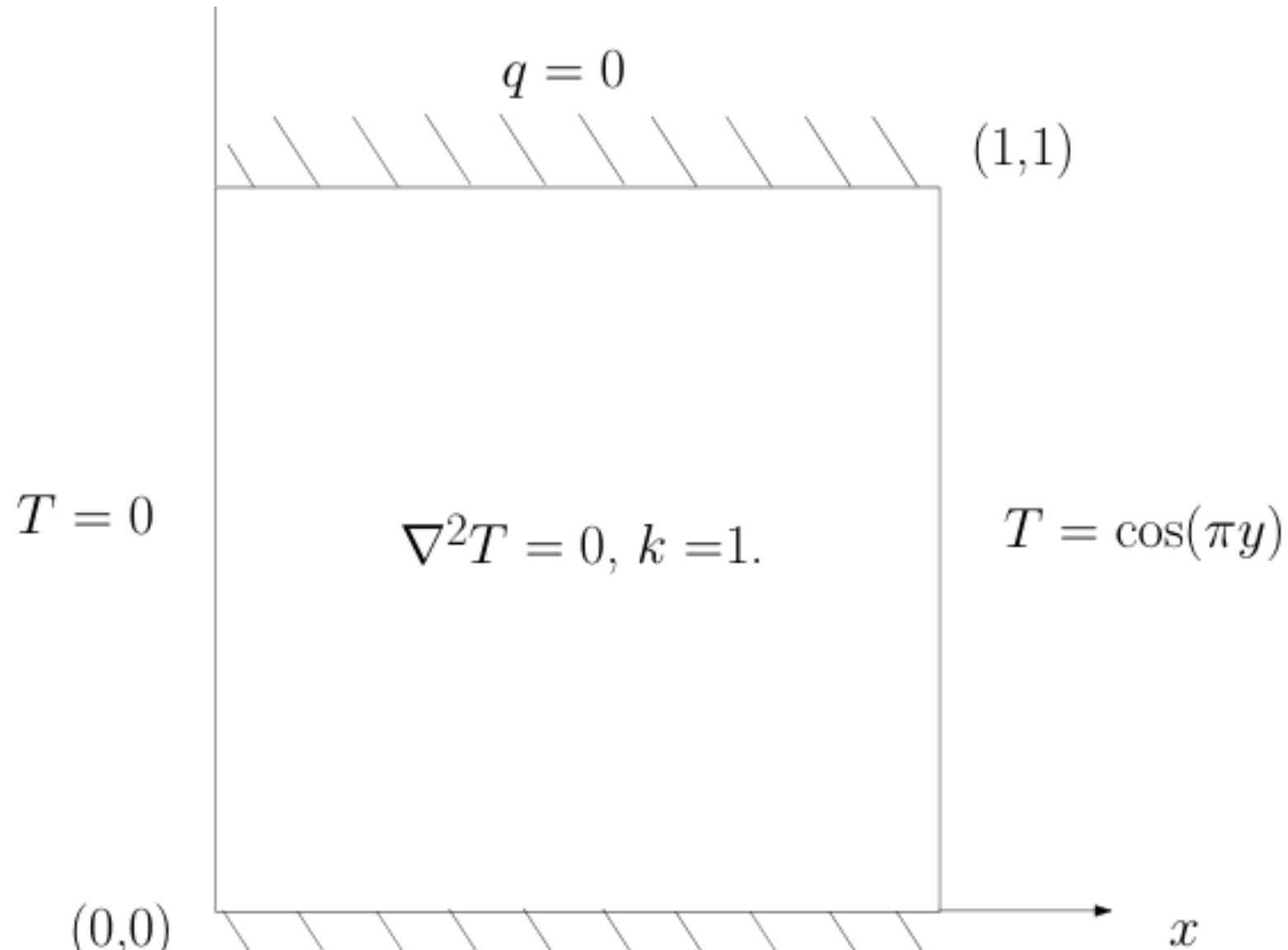
1. **E0 — convergência no quadrado.** Para `ndiv` in `{8,16,32}` (e, se puder, `tipo` in `{1,2}`), rode o lab $T = x$, monte a tabela `ndiv, tipo, N, rel_error, max|T-x|` e comente a taxa aparente.
2. **E1 — leia a malha.** Abra o trecho de grupos físicos de quadrado (ou outro `.geo` em `data/Laplace/`) e classifique cada aresta como Dirichlet ou Neumann. Confira `dad.BC / dad.BV` após `format2d`.
3. **E2 — setor circular.** Elementos lineares, quadráticos e cúbicos; erro médio, máximo e L_2 no contorno para várias malhas; gráfico de convergência. Analítico:

$$T(\theta) = \theta/\pi, \quad q(x) = -1/(\pi x) \quad \text{em} \quad y = 0.$$



1. E3 – placa mista.

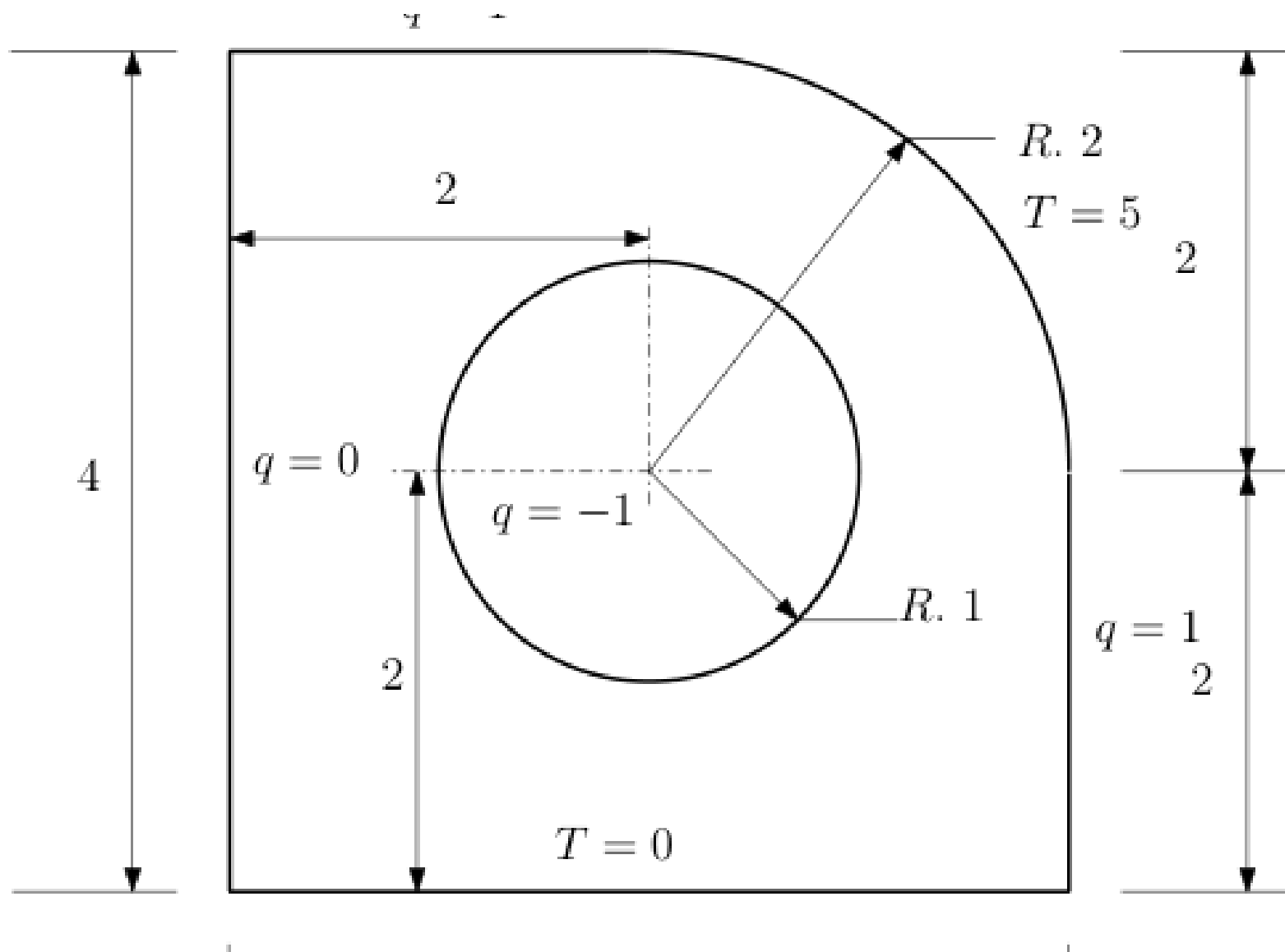
- $T(x = 0) = 0$
- $T(x = 1) = \cos(\pi y)$
- $q(y = 0) = q(y = 1) = 0$



$$T^{\text{an}} = \sinh(\pi x) \cos(\pi y) / \sinh(\pi).$$

Varie o número de elementos; tabela do erro percentual no ponto interno $(\sqrt{2}/2, \sqrt{2}/2)$.

1. **E4 — mapa de cores.** Placa da figura: obtenha T no contorno (+ internos se quiser) e produza um mapa (`plot_geo` e/ou `export` para o Gmsh). Critério: figura legível + CDCs usadas no texto.



Com base neste artigo, estime o coeficiente de sustentação de um perfil NACA via BEM. gravação

“Extra: montagem hierárquica (H-matriz)”

Fora da trilha obrigatória (também aparece em **Trabalhos finais**, proposta D). Até $N \simeq 10^3 - 5 \times 10^3$, a montagem **densa** `H_G_full_direct` basta para o curso.

Quando N cresce, memória $O(N^2)$ e fatoração direta doem. O pacote oferece

```
msh = quadrado(ndiv=80, show=false, nome="quad_fine")
dad = format2d(msh, Laplace(1.0); pontointerno=false)
attach_analytical!(dad, ana_laplace_linear(; direction=SA[1.0, 0.0]))

H_G_Hmat(dad; atol=1e-6, nmax=32)
@show compression_ratio(dad.H)
solve(dad)           # caminho iterativo quando A é hierárquica
@show rel_error(dad)
```

Critério	Densa	H-matriz
API	H_G_full_direct	H_G_Hmat
Memória	$O(N^2)$	tipicamente $O(N \log N)$
Solver	LU (bem_linsolve)	GMRES / blocos (solve_Hmat)
Curso 30 h	padrão	opcional / monografia

Regra: comece denso e grosso; só compre quando o denso não couber ou demorar demais. Detalhes: docs/src/pt-br/performance.md no repositório.

“Leituras e código”

- data/Laplace/Laplace_dad.jl — quadrado, CDCs
- src/Laplace/Fundamental.jl — T^* , q^*
- src/Core/Assembly_full.jl — H_G _full_direct, diagonal de H
- src/Core/Boundary_conditions.jl — applyBC
- src/Core/Solver.jl — solve
- src/Core/Analytical.jl — attach_analytical!, apply_analytical_bc!
- Capítulo **Indo para 2D** (geometria) · **Poisson 2D** (fonte f) · **Medidas de erro**